

```
| 1 | Edwin | Brady |
+-----+-----+
1 row in set (0.00 sec)
```

You can also do batch inserts using the *executeMany* function. For example:

```
executeMany conn "insert into users (id, fname, lname) values (?, ?, ?)" [{"2" :: String, "Simon" :: String, "Marlow" :: String},
("3" :: String, "Ulf" :: String, "Norell" :: String)]
```

You can verify the execution of the code from the *mysql* utility:

```
mysql> select * from users;
+-----+-----+-----+
| id | fname | lname |
+-----+-----+-----+
| 1 | Edwin | Brady |
| 2 | Simon | Marlow |
| 3 | Ulf | Norell |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

You can change a record entry using the *UPDATE MySQL* command:

```
execute conn "update users SET lname = 'Peyton Jones' where
fname = 'Simon'" ()
```

Executing the code in GHCi, and checking the results with the *mysql* prompt gives the following changed output:

```
mysql> select * from users;
+-----+-----+-----+
| id | fname | lname |
+-----+-----+-----+
| 1 | Edwin | Brady |
| 2 | Simon | Peyton Jones |
| 3 | Ulf | Norell |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

It is important to catch any exceptions that may arise on executing the database commands. Consider the following example, where the number of arguments passed does not match with what's expected:

```
{-# LANGUAGE OverloadedStrings #-}
import Database.MySQL.Simple
import Control.Exception
import GHC.Int
main :: IO ()
main = do
    conn <- connect defaultConnectInfo
    { connectUser = "root"
```

```
, connectPassword = "si-admin#q"
, connectDatabase = "test"
}
result <- try (execute conn "insert into users (id,
fname, lname) values (?, ?, ?)" ["4" :: String, "Laurel"
:: String]) :: IO (Either SomeException Int64)
case result of
    Left ex -> putStrLn $ "Caught exception: " ++ show ex
    Right val -> putStrLn $ "The answer was: " ++ show val
close conn
```

The error is caught when the *main* function is executed inside GHCi:

```
ghci> main
Caught exception: FormatError {fmtMessage = "3 '?' characters,
but 2 parameters", fmtQuery = "insert into users (id, fname,
lname) values (?, ?, ?)", fmtParams = ["4", "Laurel"]}
```

You can also map through the results returned from the database and use them for your needs. The following is an illustration of the same:

```
{-# LANGUAGE OverloadedStrings #-}
import Database.MySQL.Simple
import Control.Monad
import Data.Text as Text
main :: IO ()
main = do
    conn <- connect defaultConnectInfo
    { connectUser = "user"
    , connectPassword = "password"
    , connectDatabase = "test"
    }
    users <- query_ conn "SELECT fname, lname FROM users"
    forM users $ \(fname, lname) ->
        putStrLn $ Text.unpack fname ++ " " ++ Text.unpack lname
    close conn
```

The resultant output when executing the *main* function in GHCi is given below:

```
ghci> main
Edwin Brady
Simon Peyton Jones
Ulf Norell
```

You are encouraged to read *Database.MySQL.Simple* documentation from <https://hackage.haskell.org/package/mysql-simple-0.2.2.4/docs/DatabaseMySQL-Simple.html> for more information. 

By: Shakthi Kannan

The author is a free software enthusiast and blogs at shakthimaan.com